HÁZI FELADAT

Programozás alapjai 2.

Feladatválasztás/feladatspecifikáció

Hozák Krisztián Attila
F3D10S

1. Feladat

1.1. A Feladat absztraktja

1.2. A program célja és általános leírása

1.3. Adatszerkezetek és Objektummodell

1.4. A program funkciói

1.5. Felhasználói felület

1.6. Tesztelés

2. Terv

2.1. Osztályok részletes leírása

2.1.1. Adatszerkezet-osztályok

2.1.2. Fő entitások és hierarchia

1. Feladat

1.1. Sportegyesület

A Fitt Sportegyesület nyilvántartást szeretne vezetni a csapatairól. Minden csapat rendelkezik egy névvel és egy alaplétszámmal. A sportegyesület háromféle sportággal foglalkozik: labdarúgás, kosárlabda és kézilabda. A labdarúgó csapatnak két edzője van; a kosárlabda csapatnak elengedhetetlen kellékei a pom-pom lányok aminek létszámát is nyilvántartják; a kézilabda csapatok pedig évente kapnak valamekkora összegű támogatást. A nyilvántartás rendelkezzen minimum az alábbi funkciókkal: új csapat felvétele, csapat törlése, listázás. Demonstrálja a működést külön modulként fordított tesztprogrammal! A megoldáshoz ne használjon STL tárolót!

1.2. A program célja és általános leírása

A szoftver célja a Fitt Sportegyesület különböző sportágakban (labdarúgás, kosárlabda, kézilabda) működő csapatainak nyilvántartása és menedzselése. A programnak képesnek kell lennie a csapatok adatainak memóriában történő biztonságos tárolására, lekérdezésére és módosítására. Az adatok kezelése saját fejlesztésű, dinamikus memóriakezelésre épülő struktúrákban történik.

1.3. Adatszerkezetek és Objektummodell

A nyilvántartás egy objektumorientált, osztályhierarchiára épülő modellt használ. A csapatokat egy saját implementációjú dinamikus adatszerkezet tárolja, amely heterogén kollekcióként működik.

A tárolt entitások és attribútumaik:

- **Csapat (Általános ősosztály):**
  - Csapatnév (szöveges típus / dinamikus karaktertömb)
  - Alaplétszám (egész szám)
- **Labdarúgó csapat (Leszármazott):**
  - Első és második edző neve (szöveges típus)
- **Kosárlabda csapat (Leszármazott):**
  - Pom-pom lányok létszáma (egész szám)
- **Kézilabda csapat (Leszármazott):**
  - Éves támogatás összege (számszerű érték)

A feladat két dinamikus adatszerkezetet használ, egy String és egy Vektor szerkezetet. A String lehetővé teszi a könnyed karakterlánc kezelést (pl.: Nevek tárolása és ezekben való substring keresést), a Vektor pedig egy dinamikus adattároló amely lehetővé teszi az adatok memóriabiztonságos és könnyed használatát.

1.4. A program funkciói

A rendszer az alábbi alapvető műveleteket biztosítja és hiba esetén tájékoztatja a felhasználót:

- **Új csapat felvétele:** A felhasználó kiválasztja a sportágat, majd a szabványos bemeneten megadja az általános és a specifikus adatokat. Az új entitás bekerül a dinamikus tárolóba.
- **Listázás:** A nyilvántartásban szereplő összes csapat adatainak formázott megjelenítése a képernyőn.
- **Csapat törlése:** Egy kiválasztott csapat eltávolítása a nyilvántartásból (név vagy sorszám alapján), gondoskodva a dinamikusan foglalt memória memóriaszivárgás-mentes felszabadításáról.

1.5. Felhasználói felület

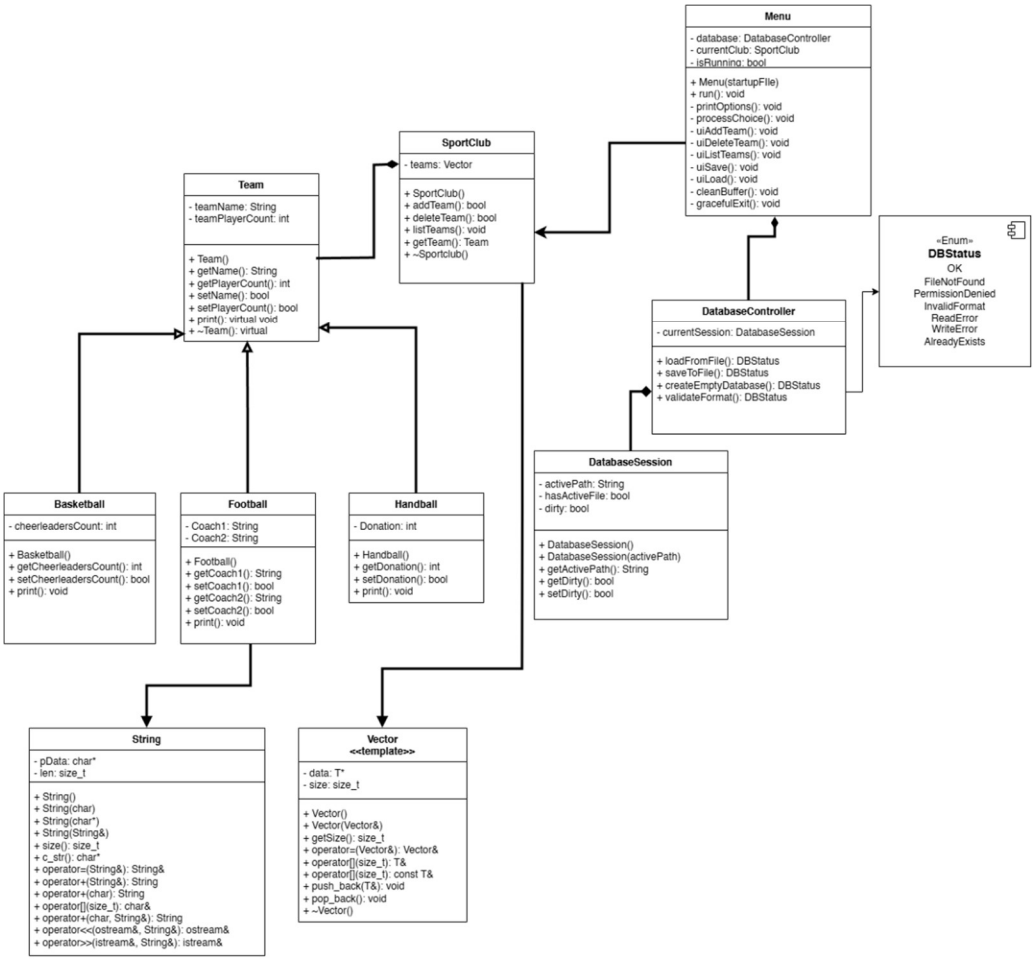
A program interaktív, konzolalapú, menüvezérelt (CLI) felülettel rendelkezik. A főmenüből átláthatóan, almenükön keresztül érhetők el az egyes funkciók, egyértelmű szöveges utasításokkal segítve a navigációt.

1.6. Tesztelés

A program architektúrailag egy olyan felépítéssel kell rendelkezzen, hogy a CLI felület “detachable” lehessen. A programot lehessen egy “flaggel” vagy bármilyen más módon elindítani, ami lehetővé teszi a unit tesztek írását. Emellett, End to end tesztek is lehetővé tehetőek ezzel, hogy maga a CLI frontend működését tesztelni lehessen.

2. Terv

A feladat megoldásához tartozó UML ábrán megvalósított terv. A *String* és a *Vector* osztályok sajátkezűleg megvalósított osztályok (könyvtárak), amelyek lehetővé teszik az egyszerűbb és átláthatóbb megvalósítást. A fájlkezeléshez és az adatok perzisztenciájához egy különálló, rétegzett architektúrát terveztem. A DatabaseSession osztály felel az éppen megnyitott adatbázisfájl állapotának nyomon követéséért. Ennek kulcsfontosságú eleme a „dirty” kapcsoló, amely azt jelzi, hogy a legutóbbi mentés óta történt-e módosítás az adatokban. Ez a flag teszi lehetővé, hogy a program kilépés előtt biztonsági visszakérdezést küldhessen a felhasználónak a nem mentett változtatásokról. A sportágspecifikus csapat osztályok (Football, Basketball, Handball) a közös Team ősosztályból származnak, kihasználva a polimorfizmus előnyeit. A mellékelt UML diagram a rendszer gerincét, illetve a minimálisan elvárt publikus interfészeket rögzíti. Az implementációs fázis során a kód tisztán tartása érdekében belső, privát segédfüggvények (helper metódusok), valamint további adathozzáférési metódusok (getter/setter függvények) bevezetése várható, azonban ezek a bemutatott globális architektúrát és az osztályok közötti kapcsolatokat nem módosítják.



2.1. Osztályok részletes leírása

2.1.1. Adatszerkezet-osztályok

A feladat saját megvalósítású dinamikus adatszerkezeteket (String, Vector) használ, elkerülve az STL tárolókat. Ezek az osztályok sajátkezű reprodukciói az alapértelmezett STL fajtájú String és Vector osztályoknak, viszont nem minden metódus van megvalósítva, hanem csak a minimum funkcionalitáshoz szükségesek.

- **String osztály**
  - **Célja:** Karakterláncok dinamikus kezelése.
- **Vector osztály**
  - **Célja:** Dinamikus, memóriabiztos adattároló, templateként, hogy bármilyen adattípusra értelmezhető legyen.

2.1.2. Fő entitások és hierarchia

A rendszer egy objektumorientált osztályhierarchiára épül, a Team ősosztályból származtatva a sportágspecifikus osztályokat.

Fájlkezelés:

- **DatabaseSession:**
  - **Célja:** Fájlkezelés és adatok perzisztenciájának biztosítása. Felelős az adatbázisfájl állapotának nyomon követéséért a dirty kapcsoló segítségével.
  - **Tagváltozók:**
    - String activePath: A sessionben módosított fájl elérési útvonala.
    - bool hasActiveFile: Igaz, ha létezik már fájl amiben módosítunk vagy egy memóriában létrehozott sportközpontot már mentettünk valahova a merevlemezre.
    - bool dirty: Igaz, ha az adatok módosultak a legutóbbi mentés óta. Lehetővé teszi a kilépés előtti visszakérdezést.
  - **Tagfüggvények:**
    - bool isDirty(): Visszatér a dirty állapot értékével.
    - void setDirty(): Beállítja a dirty állapot értékét.
    - String getActivePath(): Visszaadja a sessionben használt fájlútvonalat.

- **DatabaseController:**
  - **Célja:** A fájlkezelés minden I/O operációjáért felelős (beolvasás, írás, validálás).
  - **Tagváltozók:**
    - DatabaseSession currentSession: A jelenlegi sessionhöz tartozó databaseSession állapot.
  - **Tagfüggvények:**
    - DBStatus loadFromFile(): adatbázis betöltése fájlból
    - DBStatus saveToFile(): adatbázis elmentése fájlba
    - DBStatus createEmptyDatabase(): üres adatbázis létrehozása
    - DBStatus validateFormat(): Az adatbázis fájl formátumának a validációja.

- **DBStatus enum:**
    - **Célja:** Egy olyan Enumerációs osztály, amely a fejlesztés során segít, mivel nem statikus számokat adunk vissza hiba esetén, hanem leíró nevű “hibakódokat”. A forráskód emiatt értelmezhetőbb és olvashatóbb.

Adatmodellek:

(A getterek és a setterek nevükből adódóan egyértelmű így ezek nincsenek részletezve külön)

- **Team osztály (Általános ősosztály):**
    - **Célja:** Közös interfész biztosítása minden csapat számára.
    - **Tagváltozók:**
        - String teamName: A csapat neve.
        - Int teamPlayerCount: A csapat létszáma.
    - **Tagfüggvények:**
        - virtual void print(std::ostream& os) const = 0: Tisztán virtuális függvény a formázott listázáshoz.
- **Football osztály**
    - **Célja:** Labdarúgó csapat specifikus adatok kezelése.
    - **Tagváltozók:**
        - String coach1Name: Az első edző neve.
        - String coach2Name: A második edző neve.
    - **Tagfüggvények:**
        - void print(std::ostream& os) const override: Felülírt metódus a labdarúgó specifikus adatok megjelenítésére.
- **Basketball osztály**
    - **Célja:** Kosárlabda csapat specifikus adatok kezelése.
    - **Tagváltozók:**
        - int cheerleaderCount: A pom-pom lányok létszáma.
    - **Tagfüggvények:**
        - void print(std::ostream& os) const override: Felülírt metódus a kosárlabda specifikus adatok megjelenítésére.

- **Handball osztály**
    - **Célja:** Kézilabda csapat specifikus adatok kezelése.
    - **Tagváltozók:**
        - Int Donation: Az évente kapott támogatás összege.
    - **Tagfüggvények:**
        - void print(std::ostream& os) const override: Felülírt metódus a kézilabda specifikus adatok megjelenítésére.
- **Sportclub osztály**
    - **Célja:** A csapat osztályok kezelése (törlés, módosítás, hozzáadás, listázás)
    - **Tagváltozók:**
        - Vector teams: Vektor adatstruktúárban tárolt csapat példányok.
    - **Tagfüggvények:**
        - Bool addTeam(): csapat hozzáadása a klubhoz (visszatérési értéke a hozzáadás sikeressége, 1 ha sikerült, 0 ha nem).
        - Bool deleteTeam(): csapat törlése a klubból (visszatérési értéke a törlés sikeressége, 1 ha sikerült, 0 ha nem).
        - Void listTeams(): csapatok kilistázása.
        - Team getTeam(): csapat lekérdezése a klubból.

User Interface:

A UI megvalósításához a feladat ezen állapotakor 1 osztályal terveztem. Ez persze bővülhet, viszont a minimális funkciónalitáshoz ezen osztály elengedhetetlen.

- **Tagváltozók:**
    - DatabaseController database: Ezen példány keresztül hozza létre az adatbázist és a hozzátartozó sessiont is.
    - SportClub currentclub: az éppen szerkesztett klub példánya.
    - Bool isRunning: Kapcsoló amivel a UI működését lehet kapcsolni ki és be. Ennek az igazi felhasználása a teljes megvalósítás közben fog kirajzolódni.
- **Tagfüggvények:**
    - Void run(): publikus függvény amelyel a UI-t el lehet indítani. Azért van külön és nem a konstruktorban indul egyben, mivel a program tesztelését megkönnyíti, ha külön expliciten meg kell hívni a UI futtatását.
    - Void printOptions(): Kiírja a jelenlegi opciókat az adott módosításhoz kapcsolatosan. Ehhez a teljes implementáció elkészülésekor a teljes UI bejárási gráfja el fog készülni.

UI kisegítő privát függvények:

Ezen függvények segítik lekezelni a felhasználó által választott opciót.

- Void uiAddTeam(): a csapathozzáadáshoz kapcsolatos UI függvény.
- Void uiDeleteTeam(): a csapattörléséhez kapcsolatos UI függvény.
- Void uiListTeams(): a csapatok kilistázásához kapcsolatos UI függvény.
- Void uiSave(): az adatbázis fájlba való elmentéssel kapcsolatos UI függvény.
- Void uiLoad(): az adatbázis fájlból való betöltéssel kapcsolatos UI függvény.
- Void clearBuffer(): kisegítő függvény, a bemeneti buffer törléséhez.
- Void gracefulExit(): a kilépéskor lefutó függvény ami a felhasználó helyett is ment, egy “temporary“ fájlba, hogy megelőzzük az adatveszteséget.